

IMPLEMENTATION REPORT

Sinope — Full Localisation and Language Switching

Extracting every user-facing string into Android resources across eight languages, and repairing the language picker on Android 12.

Project	Sinope (<code>com.example.sinope</code>)
Date	29 August 2026
Scope	String externalisation + per-app locale fix
Languages	<code>en</code> · <code>ar</code> · <code>fr</code> · <code>es</code> · <code>de</code> · <code>pt</code> · <code>tr</code> · <code>ja</code>
Build status	<code>:app:assembleDebug</code> — successful
Verified on	Device — Arabic, RTL, and back to English
Device	Samsung SM-A125F, Android 12 (API 31)

1 What this covers

Two pieces of work, done in sequence.

The first was the request as given: every piece of text the user can read was hard-coded in Kotlin. It has been moved into `strings.xml` resources, one file per language listed in `AppLocale.kt`, and every call site now reads `R.string.NAME`.

The second came out of testing: the language picker did nothing on your Android 12 phone. That turned out to be a pre-existing bug unrelated to the strings work, and it is now fixed.

Both are explained below in full, including the reasoning behind each code change.

2 Part one — localisation

2.1 Where things stood

There were two resource files, `values/strings.xml` and `values-ar/strings.xml`, holding five keys between them (`app_name`, `language`, `app_language`, `language_description`, `settings`). Exactly one of them was actually used in code — `LanguageScreen.kt` line 145. Everything else the user reads was a literal in a composable.

`AppLocale.kt` mapped the `AppLanguage` enum to eight BCP-47 tags, so eight languages were the target: English, Arabic, French, Spanish, German, Portuguese, Turkish, Japanese.

2.2 The audit

Every Kotlin file under `presentation/` and `core/` was read and every string literal classified. Three groups emerged:

Group	Treatment
User-facing text	Screen titles, buttons, labels, section headers, placeholders, snackbar messages, dialog copy, content descriptions, onboarding copy. → moved to resources.
Not text at all	Compose animation labels (<code>label = "accent"</code> , <code>label = "sweep"</code> ...) — these are debug identifiers for the animation inspector, never rendered. Emoji arrays in <code>EmojiPicker</code> . The <code>"<"</code> chevron glyph. → left untouched.
Not user-facing	Room <code>@Query</code> SQL, <code>Log.d</code> tags, exception messages in <code>ParseQrCode</code> / <code>GenerateTotpCode</code> , mock placeholder values in <code>ManualEntryTab</code> (<code>"GitHub"</code> , <code>"JBSWY3DPEHPK3PXP"</code>). → left untouched.

WHY DOMAIN EXCEPTIONS WERE LEFT ALONE

`ParseQrCode` throws `IllegalArgumentException("Invalid OTPAuth URI")`. Translating that would mean injecting a `Context` into the domain layer, which breaks the layering the project already follows. These messages reach a developer through logs, not a user through the UI.

2.3 The resource files

Eight files. `values/` holds 107 keys; the seven translations hold the 104 translatable ones each — the three marked `translatable="false"` live only in the default file, as explained below.

Directory	Language	Notes
values/	English	Default — every other locale falls back here
values-ar/	Arabic	Existing file rewritten and completed
values-fr/	French	New
values-es/	Spanish	New
values-de/	German	New
values-pt/	Portuguese	New
values-tr/	Turkish	New
values-ja/	Japanese	New

Key parity was verified programmatically — every translatable key in `values/` exists in all seven translations, with no extras anywhere.

Naming

Keys are grouped by screen and named after meaning rather than location, so the same key can be reused where the same words appear. `require_on_launch`, for example, serves both the Biometric Lock row and the Language row — the original code repeated that subtitle under both, and reusing the key preserves that behaviour exactly rather than silently "fixing" it.

Where two English strings differ only in capitalisation but mean different things, they get separate keys: `edit_account` ("Edit account", the bottom-sheet action) versus `edit_account_title` ("Edit Account", the screen title). Some languages capitalise these differently, so collapsing them would have been wrong.

Strings that must not be translated

```
<string name="app_name" translatable="false">Sinope</string>
<string name="brand_wordmark" translatable="false">SINOPE</string>
<string name="secret_masked" translatable="false">.....</string>
```

These live only in `values/`. `translatable="false"` tells the build tools two things: don't warn that the key is missing from the other seven locales, and *do* raise an error if someone adds it to one. A brand name and a row of bullet characters are the same in every language.

Format arguments

Four strings carry placeholders, so the surrounding sentence can be reordered freely by each translator:

```
<string name="accounts_count">%1$d accounts</string>
<string name="seconds_left">%1$ds</string>
<string name="secret_algorithm">Secret · %1$s</string>
<string name="delete_account_dialog_title">Delete %1$s?</string>
```

Japanese, for instance, renders the last one as `%1$s を削除しますか？` — the verb moves to the end, which a concatenated string could never express.

2.4 Wiring the code — four situations

Reading a string resource in Compose means calling `stringResource(id)`, which is itself a `@Composable` function. That constraint shaped everything below.

Situation A — a literal already in composable position

The common case. Direct substitution, nothing else changes.

BEFORE — SETTINGSSCREEN.KT

```
SettingsNavRow(  
    icon = Icons.Outlined.Upload,  
    accent = SinopeColors.Green,  
    title = "Export Accounts",  
    subtitle = "Encrypted backup",  
    onClick = onExportAccounts,  
)
```

AFTER

```
SettingsNavRow(  
    icon = Icons.Outlined.Upload,  
    accent = SinopeColors.Green,  
    title = stringResource(R.string.export_accounts),  
    subtitle = stringResource(R.string.encrypted_backup),  
    onClick = onExportAccounts,  
)
```

Roughly ninety of the replacements are of this shape.

Situation B — a literal inside a non-composable lambda

An `onClick` lambda is ordinary Kotlin, not composable code. Calling `stringResource` inside it will not compile. The string has to be read *above*, while still in the composable, and captured.

BEFORE — HOMESCREEN.KT

```
QuickActionItem(  
    icon = Icons.Outlined.ContentCopy,  
    text = "Copy code",  
    onClick = {  
        scope.launch {  
            snackbarHostState.showSinopeSnackbar(  
                message = "Code copied", // not composable here  
                tone = SinopeSnackbarTone.Success  
            )  
        }  
        selectedAccount = null  
    }  
)
```

AFTER

```
// hoisted to the top of the composable, next to the other remembered state
val codeCopiedMessage = stringResource(R.string.code_copied)

// ... further down
QuickActionItem(
    icon = Icons.Outlined.ContentCopy,
    text = stringResource(R.string.copy_code),    // composable position – fine
    onClick = {
        scope.launch {
            snackbarHostState.showSnackbar(
                message = codeCopiedMessage,    // captured value
                tone = SinopeSnackbarTone.Success
            )
        }
        selectedAccount = null
    }
)
```

The same hoist was needed in five places: two in `HomeScreen`, one in `ScanQrTab` (`cameraStartFailed`, read inside `CameraX`'s `onError` callback), and one in `EditAccountScreen` (`changesSavedMessage`, read inside a `LaunchedEffect`).

WHY NOT JUST CALL GETSTRING ON A CONTEXT?

It would work, but `stringResource` is configuration-aware: if the locale changes while the screen is alive, Compose re-reads it. A value captured once from `context.getString` would go stale. Hoisting keeps that property, since the hoisted call is still inside the composable and re-runs on recomposition.

Situation C — text stored in a data class

Two data classes held display strings as plain `String`. A `String` cannot be localised after the fact — it has to become a resource *id*, resolved at the point of display.

BEFORE — LANGUAGESCREEN.KT

```
data class LanguageOption(
    val language: AppLanguage,
    val tag: String,
    val flag: String,
    val name: String,
    val nativeName: String,
)

val AppLanguages: List<LanguageOption> = listOf(
    LanguageOption(
        language = AppLanguage.FRENCH,
        tag = "fr",
        name = "French",
        nativeName = "Français",
    ),
    // ... seven more
)
```

AFTER

```

data class LanguageOption(
    val language: AppLanguage,
    val tag: String,
    val flag: String,
    @param:StringRes val nameRes: Int,
    val nativeName: String,
)

val AppLanguages: List<LanguageOption> = listOf(
    LanguageOption(
        language = AppLanguage.FRENCH,
        tag = "fr",
        nameRes = R.string.language_french,
        nativeName = "Français",
    ),
    // ... seven more
)

```

And at the point of display:

```

Text(
    text = stringResource(language.nameRes),
    color = SinopeColors.TextPrimary,
    fontSize = 13.sp,
    fontWeight = FontWeight.Bold,
)

```

Note that `nativeName` stays a `String`. It is the language's own endonym — "Français" is spelled that way regardless of what language the app is displaying, which is the whole point of showing it. Only `name`, which is the language's name *in the current UI language*, needed to become a resource. Now an Arabic user sees الفرنسية above "Français".

`@param:StringRes` puts the annotation on the constructor parameter, so Android Studio and Lint will flag anyone passing a non-string resource id.

The same change was made to `OnBoardingPage`:

```

data class OnBoardingPage(
    @param:StringRes val title: Int,
    @param:StringRes val body: Int? = null,
    val bullets: List<Int> = emptyList(),
    val art: OnboardingArt,
    val accent: Color,
)

```

This one matters more than it looks. `onboardingPages` is a top-level `val` — it is built once when the class is first loaded and never rebuilt. Had the titles stayed as `String`, they would have been frozen in whatever locale was active at process start, and a language change would never have reached the onboarding screen. Holding ids defers the lookup to `stringResource(page.title)` at draw time.

Situation D — text produced by a ViewModel

The hardest case. A ViewModel has no `Context` and must not have one, so it cannot produce a translated string. What it can do is name one.

Three files change together. First the event carries an id instead of text:

BEFORE — EDITACCOUNTUIEVENT.KT

```
data class ShowMessage(  
    val message: String,  
    val tone: SinopeSnackBarTone = SinopeSnackBarTone.Info,  
) : EditAccountUiEvent
```

AFTER

```
data class ShowMessage(  
    @param:StringRes val messageRes: Int,  
    val tone: SinopeSnackBarTone = SinopeSnackBarTone.Info,  
) : EditAccountUiEvent
```

Then the ViewModel emits ids:

```
_events.emit(  
    EditAccountUiEvent.ShowMessage(R.string.account_not_found, SinopeSnackBarTone.Error)  
)
```

And the screen resolves them, at the only place that legitimately can:

```
val context = LocalContext.current  
  
LaunchedEffect(Unit) {  
    viewModel.events.collect { event ->  
        when (event) {  
            is EditAccountUiEvent.ShowMessage ->  
                snackbarHostState.showSinopeSnackBar(  
                    context.getString(event.messageRes),  
                    event.tone,  
                )  
            // ...  
        }  
    }  
}
```

`context.getString` rather than `stringResource` here because a `LaunchedEffect` body is a coroutine, not composable code — same constraint as Situation B, resolved the same way.

The identical change was applied to `AddAccountUiEvent.AccountAlreadyExists`.

Form validation state

`EditAccountState` computed its own error text:

BEFORE

```
val issuerError: String? =  
    if (!isLoading && issuer.isBlank()) "Issuer can't be empty" else null
```

AFTER

```
@get:StringRes  
val issuerError: Int? =  
    if (!isLoading && issuer.isBlank()) R.string.issuer_cant_be_empty else null
```

This one is neat, because `issuerError` was already used in two very different ways and both keep working. The screen renders it:

```
state.issuerError?.let { error ->
    Text(text = stringResource(error), color = SinopeColors.Danger, ...)
}
```

...and the `ViewModel` falls back through it when a save is refused:

```
EditAccountUiEvent.ShowMessage(
    current.issuerError ?: R.string.nothing_to_save,
    SinopeSnackbarTone.Error,
)
```

Because both the error and the fallback are now `Int` resource ids, the elvis operator still type-checks and the logic is unchanged.

ONE DELIBERATE BEHAVIOUR CHANGE

The save and delete failure paths previously surfaced the raw exception text:

```
error.message ?: "Couldn't save changes"
```

A `Throwable.message` comes from the JVM or from Room and is never localised, so it cannot survive this refactor. Those paths now always show `R.string.couldnt_save_changes` / `R.string.couldnt_delete_account`. Users get a sentence in their own language; the underlying exception is still available in logcat. If you would rather keep the raw text visible for debugging, it can be logged explicitly before the event is emitted.

An idiom worth pointing out

```
Text(text = issuer.ifBlank { stringResource(R.string.untitled) })
```

This compiles even though `ifBlank` takes a lambda, because `ifBlank` is declared `inline`. Kotlin inlines the lambda body into the enclosing composable, so the `stringResource` call is still in composable position. The same trick is used for `no_account_name` and `this_account`.

2.5 Result

After the pass, a search for user-facing literals across `presentation/` and `core/` returns only: animation labels, the emoji palette, the "<" glyph, and one "back" inside a `@Preview` function that never ships. Nothing a user can read is hard-coded.

3 Part two — the Android 12 bug

3.1 Symptom

On the SM-A125F (Android 12, API 31), picking a language in the app moved the radio button and survived a restart, but the interface stayed in English. On Android 13 and newer the same code worked.

3.2 Root cause

The entire body of `applyLanguages` sat inside a version check:

```
fun applyLanguages(context: Context, language: AppLanguage) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        val localeManager = context.getSystemService(LocaleManager::class.java)
        val languageTag = when (language) { /* ... */ }
        localeManager.applicationLocales = LocaleList.forLanguageTags(languageTag)
    }
    // no else branch
}
```

TIRAMISU is API 33. On API 31 the condition is false, there is no `else`, and the function returns having done nothing at all.

The save still happened — `LanguageViewModel.selectLanguage` calls `saveLanguage(language)` before `applyLanguages` — which is exactly why the radio button appeared to work. The choice was recorded and never applied.

This was not caused by the localisation work; the resource files were verified correct by switching the phone's own system language, which the app followed properly. And because `minSdk = 30`, the bug affected API 30, 31 and 32 alike.

3.3 The route that did not work

The standard answer is `AppCompatActivity.setApplicationLocales`, AndroidX's backport, which works from API 21 and forwards to the framework `LocaleManager` on 33+. That was implemented first. It requires the activity to be an `AppCompatActivity`, because below API 33 the backport applies locales by walking its registry of AppCompatActivity delegates and recreating them — a plain `ComponentActivity` is never in that registry.

Changing `MainActivity`'s superclass broke the build:

```
e: [ksp] MainActivity.kt:25: [Hilt] @AndroidEntryPoint base class must extend
ComponentActivity, (support) Fragment, View, Service, or BroadcastReceiver.
```

`AppCompatActivity` does extend `ComponentActivity`, through `FragmentActivity`. To isolate the failure, `MainActivity` was temporarily made a `FragmentActivity` — which Hilt accepted without complaint. So the supertype chain resolves for `FragmentActivity` but not through `AppCompatActivity`: a resolution defect in the KSP 2 / Hilt 2.60.1 combination, not in your code.

Rather than fight the annotation processor, that whole approach was reverted. `MainActivity` stays a `ComponentActivity`, the theme was restored to `android:Theme.Material.Light.NoActionBar`, and the manifest service entry was left as it was.

3.4 What was implemented instead

Applying a locale without AppCompatActivity means doing directly what AppCompatActivity does internally: build a `Configuration` carrying the locale, hand it to the activity before it inflates anything, and recreate the activity when the choice changes.

`core/Utils/AppLocale.kt` was rewritten. Walking through it:

The tag mapping, extracted

```
fun AppLanguage.toLanguageTag(): String = when (this) {
    AppLanguage.SYSTEM -> ""

    AppLanguage.ENGLISH -> "en"
    AppLanguage.ARABIC -> "ar"
    AppLanguage.FRENCH -> "fr"
    AppLanguage.SPANISH -> "es"
    AppLanguage.GERMAN -> "de"
    AppLanguage.PORTUGUESE -> "pt"
    AppLanguage.TURKISH -> "tr"
    AppLanguage.JAPANESE -> "ja"
}
```

This was inline in the old function. It is now an extension on the enum because two separate code paths need it. An empty tag means "follow the system", which is what `AppLanguage.SYSTEM` represents.

The entry point

```
fun applyLanguages(
    context: Context,
    language: AppLanguage,
) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        context.getSystemService(LocaleManager::class.java).applicationLocales =
            LocaleList.forLanguageTags(language.toLanguageTag())
    } else {
        context.findActivity()?.recreate()
    }
}
```

The 33+ branch is deliberately unchanged from your original. It already worked, it is the platform-blessed path, and it registers the choice with the operating system so the app appears in Settings → Apps → Sinope → Language. There was no reason to replace behaviour that was already correct.

The `else` branch is the new part, and it is one line, because the language has *already been saved* by the time this runs — `selectLanguage` awaits `saveLanguage(language)` first. All that remains is to restart the activity so it picks the new value up on the way in.

Applying the locale on the way in

```
fun Context.withPersistedAppLocale(): Context {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) return this

    val tag = readPersistedLanguage().toLanguageTag()
    if (tag.isEmpty()) return this

    val locale = Locale.forLanguageTag(tag)
    Locale.setDefault(locale)

    val configuration = Configuration(resources.configuration).apply {
        setLocale(locale)
        setLayoutDirection(locale)
    }
    return createConfigurationContext(configuration)
}
```

Line by line:

- On API 33+ this returns immediately — the framework has already applied the per-app locale before any context is handed out, and re-applying it would be redundant.
- An empty tag means the user chose *System*, so the context is returned untouched and the device language wins.
- `Locale.setDefault` sets the process-wide default, so anything going through `java.util` — number and date formatting, `String.format`, `toUpperCase` — agrees with the UI.
- The `Configuration` is *copied* from the current one, not built from scratch, so screen density, font scale, dark-mode state and everything else are preserved; only the locale is overridden.
- `setLayoutDirection(locale)` is what makes Arabic lay out right-to-left. The manifest already declares `android:supportsRtl="true"`, so Compose reads the direction from this configuration and mirrors the whole tree.
- `createConfigurationContext` returns a new context whose `Resources` resolve against that configuration — which is precisely what `stringResource` reads from.

Reading the saved value

```
private fun Context.readPersistedLanguage(): AppLanguage = runCatching {
    runBlocking {
        val stored = datastore.data.first()[stringPreferencesKey(Constants.LANGUAGE)]
        AppLanguage.valueOf(stored ?: AppLanguage.SYSTEM.name)
    }
}.getOrElse(AppLanguage.SYSTEM)
```

`DataStore` is asynchronous and `attachBaseContext` cannot suspend, so this blocks. That is a considered trade-off rather than an oversight:

- The alternative is a second, synchronous store (`SharedPreferences`) mirroring the same value, which introduces two sources of truth that can drift apart.
- It is a single small preferences read, on a file the app opens at startup regardless.
- It only runs on API < 33 — on newer devices the function returns before reaching here.
- It happens while the splash screen is up, which `MainViewModel` already holds open until `readAppEntry()` emits.

`runCatching { ... }.getOrDefault(AppLanguage.SYSTEM)` means a corrupt or unreadable preference degrades to the device language instead of crashing at startup — a failure here would otherwise take down the app before `onCreate` ever ran.

Finding the activity

```
private tailrec fun Context.findActivity(): Activity? = when (this) {  
    is Activity -> this  
    is ContextWrapper -> baseContext.findActivity()  
    else -> null  
}
```

The `context` that reaches `applyLanguages` comes from `LocalContext.current` in `LanguageScreen`, and Compose does not necessarily hand back the activity itself — it may be a `ContextWrapper` around it. This unwraps the chain until it finds one. `tailrec` lets the compiler turn the recursion into a loop, so a deep wrapper chain cannot overflow the stack.

The hook in MainActivity

```
@AndroidEntryPoint  
class MainActivity : ComponentActivity() {  
  
    private val viewModel by viewModels<MainViewModel>()  
  
    // Below API 33 there is no per-app language support in the platform, so the  
    // saved locale has to be folded into the activity's context here – before  
    // onCreate inflates anything.  
    override fun attachBaseContext(newBase: Context) {  
        super.attachBaseContext(newBase.withPersistedAppLocale())  
    }  
  
    // ... onCreate unchanged  
}
```

`attachBaseContext` is the earliest point in an activity's life where its `Context` can be substituted — it runs before `onCreate`, before the splash screen, before Compose exists. Anything resolved later, including every `stringResource` in the tree, resolves against the context installed here.

The full cycle

1. User taps العربية in the language list.
2. `LanguageViewModel.selectLanguage` awaits `saveLanguage(ARABIC)`; `DataStore` now holds "ARABIC".
3. `applyLanguages` runs. On API 31 it takes the `else` branch and calls `recreate()`.
4. Android tears the activity down and builds a fresh one. `attachBaseContext` runs first.
5. `withPersistedAppLocale` reads "ARABIC" back out of `DataStore`, builds a configuration with locale `ar` and RTL layout direction, and returns the re-based context.
6. `onCreate` runs, Compose renders, every `stringResource` resolves against `values-ar/strings.xml`, and the layout mirrors.

3.5 Right-to-left layout

With the locale applied, Arabic strings appeared — but the layout stayed left-to-right. Flags sat on the left, titles on the left, chevrons on the right. Correct words, wrong direction.

Cause

`createConfigurationContext` overrides *resource lookup*. It does not override the window. Dumping the running activity showed its window configuration still carrying the device's locale list and an explicit left-to-right marker:

```
overrideConfig={1.0 604mcc1mnc [en_US,de_DE,fr_FR] ldltr sw384dp w384dp h781dp 300dpi ...}
```

Compose derives `LocalLayoutDirection` from the layout direction resolved by the host `View`, and that resolves from the window configuration `ViewRootImpl` applies — the system-assigned one above, not the overridden context. So resources came from the Arabic context while direction came from the English window.

Fix

Provide the direction into the composition explicitly, derived from the locale that is actually in effect:

```
setContent {
    // The window's own configuration still carries the device's layout direction – an
    // overridden Context changes resource lookup, not the window. Derive the direction
    // from the active locale so Arabic lays out right-to-left.
    val locale = ConfigurationCompat.getLocales(LocalConfiguration.current)[0]
    val layoutDirection = if (
        locale != null &&
        TextUtils.getLayoutDirectionFromLocale(locale) == View.LAYOUT_DIRECTION_RTL
    ) {
        LayoutDirection.Rtl
    } else {
        LayoutDirection.Ltr
    }

    CompositionLocalProvider(LocalLayoutDirection provides layoutDirection) {
        SinopeTheme {
            // ... unchanged
        }
    }
}
```

- `LocalConfiguration` reflects the *overridden* context, so it names the chosen language rather than the device's.
- The direction is computed from the locale rather than read out of `Configuration.layoutDirection`, so it does not depend on that field surviving the context override.
- Every left-to-right language yields `LayoutDirection.Ltr`, which is Compose's default — nothing changes for them.
- `ModalBottomSheet`, `AlertDialog` and the snackbar read `LocalLayoutDirection` from the composition rather than from their own windows, so they mirror too, despite living in separate windows.
- On API 33+ the framework sets the window configuration itself, and the value computed here matches what Compose would have derived anyway — so the same code is correct on both paths.

A DETAIL WORTH NOTICING

On the device the account count renders as حساب ١ and the countdown as ٣ ث — Arabic-Indic digits. That comes free from `stringResource(R.string.accounts_count, count)` formatting the number against the active locale. A string built with Kotlin interpolation, `"$count accounts"`, would have printed Western digits inside Arabic text.

3.6 A dependency defect found along the way

gradle/libs.versions.toml declared:

```
appcompat = "<your-version>"
```

That is a template placeholder, not a version. The build only worked because ML Kit pulls `androidx.appcompat` in transitively and Gradle's conflict resolution quietly substituted the real thing:

```
\--- androidx.appcompat:appcompat:<your-version> -> 1.7.0
```

The day ML Kit stopped depending on `appcompat`, the build would have failed with an unresolvable version. Pinned to `1.7.0`, the version actually in use.

4 Files changed

Resources

File	Change
res/values/strings.xml	Rewritten — 107 keys (104 translatable)
res/values-ar/strings.xml	Rewritten — full Arabic
res/values-{fr,es,de,pt,tr,ja}/strings.xml	New — six files

Localisation — Kotlin

File	Change
presentation/home/HomeScreen.kt	10 strings; 2 hoists
presentation/home/components/SinopeHeader.kt	Wordmark, account count
presentation/home/components/AccountCard.kt	Copy label, countdown, clipboard label
presentation/settings/SettingsScreen.kt	18 strings
presentation/language/LanguageScreen.kt	LanguageOption.name → nameRes
presentation/onboarding/OnBoardingPage.kt	Fields → @StringRes Int
presentation/onboarding/OnBoardingScreen.kt	Page defs + nav buttons
presentation/addaccount/AddAccountScreen.kt	Title, tabs, event resolution
presentation/addaccount/components/ManualEntryTab.kt	5 strings
presentation/addaccount/components/ScanQrTab.kt	15 strings; 1 hoist
presentation/addaccount/viewModel/AddAccountUiEvent.kt	→ @StringRes messageRes
presentation/addaccount/viewModel/AddAccountViewModel.kt	Emits resource id
presentation/editAccount/EditAccountScreen.kt	20 strings; 1 hoist
presentation/editAccount/components/ {AccountIdentityCard,DeleteAccountDialog,FavoriteRow}.kt	8 strings
presentation/editAccount/viewModel/EditAccountUiEvent.kt	→ @StringRes messageRes
presentation/editAccount/viewModel/EditAccountState.kt	issuerError → Int?
presentation/editAccount/viewModel/ EditAccountViewModel.kt	Emits resource ids
core/common/{SinopeButton,SinopeSnackbar}.kt	2 strings

Language-switching fix

File	Change
core/utils/AppLocale.kt	Rewritten — dual path, context re-basing
mainActivity/MainActivity.kt	attachBaseContext override; LocalLayoutDirection provider for RTL
gradle/libs.versions.toml	appcompat version pinned

Untouched: the data layer, the domain layer, dependency injection, navigation, themes and colours, and the manifest.

5 Verification

Check	Result
Resource key parity across 8 locales	Passed — the same 104 translatable keys in all eight files, no gaps, no extras
XML well-formedness, all 8 files	Passed
:app:assembleDebug	Passed
Resource merge / processDebugResources	Passed — all locales compiled into the APK
Install on SM-A125F (API 31)	Passed
On-device switch to Arabic	Passed — Home, Settings and Language screens all render Arabic
Right-to-left mirroring	Passed — all three screens mirror; verified by screenshot
Switch back to English	Passed — reverts to English and left-to-right immediately

Both language switching and right-to-left layout were confirmed on the Android 12 handset itself, not merely reasoned about: the app was driven through the picker, screenshotted in Arabic and in English, and the accessibility tree inspected to confirm element positions had actually mirrored rather than merely looking different.

6 Recommendations

Plural forms

accounts_count is a plain string, "%1\$d accounts", so it reads "1 accounts" for a single account. The correct construct is `<plurals>`, which lets each language declare its own quantity rules — Arabic needs six forms, French three, Japanese one. It was left as a string to match the `R.string` pattern you asked for. Converting it is a contained change, one resource block per locale plus one call site.

System language settings integration

There is no `res/xml/locales_config.xml` and no `android:localeConfig` in the manifest. The in-app picker works without it, but Android's own *Settings* → *Apps* → *Sinope* → *Language* screen will not list Sinope's languages until it exists. Adding it is a small XML file plus one manifest attribute, and it also helps the Play Store advertise which languages the app supports.

The Hilt / AppCompatActivity defect

Worth revisiting when Hilt or KSP is next upgraded. If `@AndroidEntryPoint class MainActivity : AppCompatActivity()` starts compiling, the custom locale code in `AppLocale.kt` could be replaced by `AppCompatActivity.setApplicationLocales` — fewer moving parts, and it removes the blocking `DataStore` read. Nothing is broken as it stands; it is simply less code to own.

Translation review

The seven translations are careful but machine-produced. Before release, a native reader should check tone and terminology per language — particularly the security vocabulary ("vault", "secret", "issuer"), which carries connotations that vary by locale.